

# GPLA: Robust and Dynamic Piecewise Linear Approximations for Learned Indexes

Anonymous Author(s)

## Abstract

- Neurips: Sydney Australia Dec 2026  
Deadline mid May
- FOCS: New York, November 2026  
Deadline Early April

TODO points of comparison with ALEX:

- (1) Iterating (i.e repeated successors).
  - (2) Removing half of the data at random, and then doing iterations.
  - (3) range queries
- Types of robustness:
- (1) Permutation.
  - (2) Sampling.
  - (3) Dead zone deletions.
  - (4) Uniform random deletions.
  - (5) Against fuzzy.

## Keywords

learned indexes, piecewise linear approximation, dynamic indexes

### ACM Reference Format:

Anonymous Author(s). 2026. GPLA: Robust and Dynamic Piecewise Linear Approximations for Learned Indexes. In . ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

## 1 Introduction

Learned indexes have emerged as a promising approach to ordered indexing that exploit regularity in the stored data to reduce the cost of search [4, 6? ]. Rather than locating a query solely through a generic comparison-based search structure, a learned index learns a model of the key distribution and uses it to narrow the query to the region in which its answer is expected to lie, followed by a local corrective search. When data is sufficiently regular, much of the work performed by a traditional index can consequently be replaced by a small learned representation and a correspondingly small search region. The potential gains are substantial: learned indexes have demonstrated several-fold improvements in lookup throughput over traditional indexes on favorable datasets, while in some settings reducing auxiliary index space by multiple orders of magnitude [4? ? ? ]. [Here we will include either a forward reference for more description of existing

learned indexes and literature review, or refer the reader to an existing paper for a state-of-the-art review, that is not our point here]. The attraction of learned indexing is fundamentally data-dependent: a learned index can capitalize on structured data in ways that generic indexes cannot. However, this same dependence raises a natural tension in designing learned indexes, identified in the original paper by [4]: The authors ask whether, as the data distribution changes, learned indexes could provide “similar strong guarantees as B-Trees which always guarantee  $O(\log n)$  look-up and insertion costs”. This question captures an appealing high-level goal for learned index design: Exploit regularity in the data when doing so helps, without sacrificing the worst-case safety of a traditional index when it does not. Subsequent work has highlighted this tension in progressively stronger forms: learned indexes can deteriorate substantially more than traditional indexes as the data evolves; this deterioration can erase their performance advantage over traditional indexes altogether, and, in the absence of worst-case guarantees, deliberately unfavorable updates can cause deterioration to become effectively unbounded. In a broad evaluation by [7], experiments that change the indexed data distribution over time show substantially greater performance deterioration for leading learned indexes than for traditional indexes. ALEX loses up to 52% of its throughput when moving from an easy to a hard distribution, and LIPP exhibits similar degradation, while traditional indexes remain comparatively stable. Crucially, this deterioration is not merely relative, but in various cases carries performance below the traditional-index baseline: the same study identifies workload regimes in which a traditional index outperforms every evaluated learned index. In particular, updatable learned indexes fail to outperform traditional indexes on hard datasets with at least 50% writes. More recent work has further quantified the extent of this deterioration. [5] systematically evaluate state-of-the-art updatable learned indexes across various datasets and dynamic workloads and conclude that “none of the state-of-the-art updatable learned indexes can robustly outperform traditional indexes.” In their single-threaded lookup experiments, updatable learned indexes perform worse than ART in 44.3% of tested cases and worse than B+-tree in 10.9%. The worst measured lookup throughput of ALEX and LIPP falls 35.4% and 86.9% below ART, respectively. The authors describe these robustness issues as “hindering their broader adoption.” An independent recent study identifies the same deployment concern: [9] argues that “performance uncertainty would make practitioners reluctant to deploy learned indexes in production systems”. While the aforementioned studies characterize performance deterioration under ordinary workloads, [8] investigates its worst-case extent. They observe that several mechanisms used to make ALEX dynamic and adaptive lack worst-case guarantees. Adversarially chosen insertion sequences can force pathological structural behavior, including severe throughput degradation and memory exhaustion. The authors conclude by explicitly “stressing the need for new

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).  
*Conference'17, Washington, DC, USA*

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.  
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM  
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

designs of dynamic learned indexes with worst-case guarantees.”. Recent work has begun to directly target this weakness in learned indexes [forward reference here for discussing potentially HIRE, together with LIPP’s alleged worst-case guarantees which do not accommodate deletions, or should that be a footnote, and maybe Eva’s work]. However, the original question posed by [4] of whether learned indexes can retain worst-case guarantees comparable to traditional indexes remains open. We resolve this question by introducing the first fully dynamic learned index with worst-case guarantees of  $O(\log(n))$ -time per query, comparable to traditional indexes. Specifically, the running time per query is  $O(\log \lambda + \log \Delta)$  where  $\Delta \in O(n)$  is the prescribed prediction error, and  $\lambda \in O(n)$  is the data-dependent complexity of the learned model. It thus exploits regularity when it is present, while retaining the worst-case safety of a traditional index when it is not.

## 1.1 The challenge of dynamizing learned indexes

At a high level, learned indexing couples two objects: a memory layout, which assigns stored keys to physical positions, and a learned model, typically a PLA, which predicts these positions from the keys. Dynamization therefore requires more than making a static model updatable. The goal is to find a layout that can itself be maintained efficiently under insertions and deletions, while simultaneously maintaining a compact and accurate model of the resulting key-to-position mapping. A successful dynamization should preserve the central learned-index promise - when the data admits a simpler model, we should obtain faster query times, and a more succinct structure.

**1.1.1 Dynamic PGM.** The first proposed dynamic learned index with worst-case guarantees was introduced by [2]. They overcome the need to explicitly maintain a coupled layout-model pair by applying the Bentley-Saxe [1] method for generically dynamizing their static structure. The method partitions the data among  $O(\log n)$  independent static structures of geometrically increasing sizes. There are two central issues with this approach for dynamization. First and more fundamentally, as noted by [3], this approach introduces an additional  $O(\log n)$  factor in query time. This overhead persists even on perfectly learnable data. Consider equally spaced keys inserted in sorted order, with  $n = 2^r - 1$  for some  $r > 0$ . All  $r = \Theta(\log n)$  static structures are occupied. Thus, although the static learned index answers queries in  $O(1)$  time for constant error, its dynamic counterpart requires querying  $\Theta(\log n)$  independent models yielding  $\Omega(\log n)$  query time. Hence, this method of dynamization can erase precisely the asymptotic advantage that learning is intended to provide: even an exactly linear rank function degenerates to comparison-tree-like logarithmic query time.

Deletions introduce a second pathology. PGM represents deleted elements by tombstones rather than removing them from the corresponding static structures. As observed by [3], this destroys output sensitivity: a range containing only  $k$  live elements may contain  $\Theta(N)$  stale deleted elements, where  $N$  is the maximum historical size, forcing  $\Theta(N)$  work independently of  $k$ .

**1.1.2 Eva’s approach.** Gæde et al. [3] manage to dynamically maintain a single learned model with worst-case guarantees. However, their formulation introduces a different fundamental instability: the learned complexity of an instance depends on the numerical scale of the key domain. A notion of linear learnability should, at a minimum, be invariant under scaling, the simplest linear transformation of keys. Multiplying every key by a constant changes only the units in which the input domain is represented; it does not make the underlying key-to-position relationship intrinsically more difficult to approximate linearly. Their formulation violates this basic invariance. The issue stems from what they term an  $\varepsilon$ -cover which is used to support queries. The standard notion for a linear model maps keys to ranks, and measures error by the distance between a predicted position and the correct position of a key. Their  $\varepsilon$ -cover instead maps ranks to keys and measures error directly in key-space. Consider the key-set  $S = \{s_i = 4i + (-1)^i\}$ . A permitted error of  $\varepsilon = 1$  allows modeling the (rank, key) pairs using just a single line  $y = 4x$ , and results in an index which supports queries in constant time. Now, consider the scaled instance  $S_c = \{c \cdot s_i\}$  for some  $c > \varepsilon$ . The approximation problem now changes drastically: Within the permitted error no single line can cover three consecutive points, and thus the model complexity blows up to  $\Omega(n)$ . One can try to control the model complexity by increasing the permitted error,  $\varepsilon$  to be greater than  $c$ . However, due to their partition of key space into strips of width  $\varepsilon$ , their query time depends **linearly** on  $\varepsilon$ . So, increasing the permitted error will degrade query time even if it simplifies the underlying model. Specifically, setting  $c \geq \log(n)$  causes their structure’s query time for  $S_c$  to degrade to the trivial  $\Omega(\log(n))$  for any value of  $\varepsilon$ . The issue is not that model complexity and query time trade off—such a tradeoff is standard in learned indexing. Rather, in their formulation the entire tradeoff is representation-dependent: a simple rescaling of the key domain can arbitrarily worsen the achievable combinations of model size and query time, despite leaving the underlying indexing problem unchanged.

## 2 Preliminaries

For a set of keys  $S \subseteq U$  a BGPLA is a piecewise linear approximation of the values of some monotone function  $f(\cdot)$  on  $S$  with bounded gaps. We proceed to provide formal definitions, and basic properties of a GPLA.

**Definition 2.1.** Let  $S \subseteq U$  be a set of keys,  $f(\cdot)$  be a monotone function on  $U$ . A set of triplets  $\mathcal{L} = \{(a_1, m_1, b_1), (a_2, m_2, b_2), \dots, (a_\lambda, m_\lambda, b_\lambda)\}$  a **generalized piecewise linear approximation (GPLA) of  $S$  with error  $\delta$**  (with respect to  $f$ ) if and only if the following conditions hold:

- (1)  $a_1 < a_2 < \dots < a_\lambda$ .
- (2) For every  $i \in [1, \lambda]$ ,  $m_i > 0$ .
- (3) For every  $i \in [1, \lambda]$ , for every  $k \in S_i := S \cap [a_i, a_{i+1})$  (letting  $a_{\lambda+1}$  be  $\infty$ ) we have:

$$|m_i \cdot k + b_i - f(k)| < \delta.$$

Each line segment in a GPL is identified by a triplet including its starting point  $a_i$ , its slope,  $m_i > 0$ , and an intersect point. Each such line covers a certain interval of keys. More precisely, the line corresponding to  $(a_i, m_i, b_i)$  provides a  $\delta$ -approximation for the

values of  $f$  on all keys in the set  $S \cap [a_i, a_i + 1)$ , which we denote by  $S_i$ . We proceed to bound the approximation a GPLA achieves for keys outside of our set. Specifically, we can obtain a meaningful bound when such a key has both a successor and a predecessor in a certain interval of keys, as defined below. Note that when we describe a GPLA without explicit mention of  $\delta, f$ , it is either clear from context, or the specific parameters are unimportant.

**Definition 2.2.** Let  $S \subseteq U$  be a set of keys and let  $\mathcal{L} = \{(a_1, m_1, b_1), (a_2, m_2, b_2), \dots, (a_\lambda, m_\lambda, b_\lambda)\}$  be a GPLA of  $S$ . We say that  $k \in U$  is internal to  $\mathcal{L}$  if and only if there exists  $i \in [1, \lambda]$  such that  $\min S_i \leq k \leq \max S_i$  (letting  $a_{\lambda+1}$  be  $\infty$ ).

**Lemma 2.3.** Let  $S \subseteq U$  be a set of keys,  $f(\cdot)$  be a monotone function on  $U$ , let  $\mathcal{L} = \{(a_1, m_1, b_1), (a_2, m_2, b_2), \dots, (a_\lambda, m_\lambda, b_\lambda)\}$  be a GPLA of  $S$  with error  $\delta$  (with respect to  $f$ ), and let  $k \in U \setminus S$  be internal to  $\mathcal{L}$ , i.e.,  $\min S_i \leq k \leq \max S_i$ . Then,  $|m_i \cdot k + b_i - f(k)| < f^* + \delta$  where  $f^*$  is the maximum difference between the value of  $f$  on two consecutive keys in  $S$ .

**PROOF.** Proof. Since  $\min S_i \leq k \leq \max S_i$  it has a successor and predecessor in  $S_i$ , denote these elements by  $\text{pred}(k)$  and  $\text{succ}(k)$ . Note that:

$$m_i \cdot \text{pred}(k) + b_i \leq m_i \cdot k + b_i \leq m_i \cdot \text{succ}(k) + b_i.$$

Since  $\mathcal{L}$  is a GPLA with error  $\delta$  this implies:

$$f(\text{pred}(k)) - \delta \leq m_i \cdot k + b_i \leq f(\text{succ}(k)) + \delta.$$

Since  $f$  is monotone, we have:

$$f(\text{pred}(k)) \leq f(k) \leq f(\text{succ}(k)).$$

Therefore, we have:

$$\begin{aligned} m_i \cdot k + b_i - f(k) &\leq f(\text{succ}(k)) - f(\text{pred}(k)) + \delta \text{ and} \\ m_i \cdot k + b_i - f(k) &\geq -(f(\text{succ}(k)) - f(\text{pred}(k)) + \delta) \end{aligned}$$

So, we can conclude:

$$|m_i \cdot k + b_i - f(k)| \leq f(\text{succ}(k)) - f(\text{pred}(k)) + \delta = f^* + \delta.$$

□

## References

- [1] Jon Louis Bentley and James B Saxe. 1980. Decomposable searching problems I. Static-to-dynamic transformation. *Journal of Algorithms* 1, 4 (Dec. 1980), 301–358. doi:10.1016/0196-6774(80)90015-2
- [2] Paolo Ferragina and Giorgio Vinciguerra. 2020. The PGM-index: a fully-dynamic compressed learned index with provable worst-case bounds. *Proc. VLDB Endow.* 13, 8 (2020), 1162–1175. doi:10.14778/3389133.3389135
- [3] Emil Toftgaard Gæde, Ivor van der Hoog, Eva Rotenberg, and Tord Stordalen. 2025. A Dynamic Piecewise-Linear Geometric Index with Worst-Case Guarantees. In *33rd Annual European Symposium on Algorithms, ESA 2025, Warsaw, Poland, September 15–17, 2025 (LIPIcs, Vol. 351)*, Anne Benoit, Haim Kaplan, Sebastian Wild, and Grzegorz Herman (Eds.). Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 64:1–64:18. doi:10.4230/LIPICS.ESA.2025.64
- [4] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. 2018. The Case for Learned Index Structures. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD '18)*. Association for Computing Machinery, New York, NY, USA, 489–504. doi:10.1145/3183713.3196909
- [5] Yuanhui Luo, Minhui Xie, Yiheng Tong, Shichao Jiang, and Yunpeng Chai. 2025. Understanding Robustness Issues of Updatable Learned Indexes: [Experiments & Analysis]. *Proc. ACM Manag. Data* 3, 4 (2025), 270:1–270:25. doi:10.1145/3749188
- [6] Zhaoyan Sun, Xuanhe Zhou, and Guoliang Li. 2023. Learned Index: A Comprehensive Experimental Evaluation. *Proc. VLDB Endow.* 16, 8 (April 2023), 1992–2004. doi:10.14778/3594512.3594528
- [7] Chaichon Wongkham, Baotong Lu, Chris Liu, Zhicong Zhong, Eric Lo, and Tianzheng Wang. 2022. Are Updatable Learned Indexes Ready? *Proc. VLDB Endow.* 15, 11 (2022), 3004–3017. doi:10.14778/3551793.3551848
- [8] Rui Yang, Evgenios M. Kornaropoulos, and Yue Cheng. 2023. Algorithmic Complexity Attacks on Dynamic Learned Indexes. *Proc. VLDB Endow.* 17, 4 (2023), 780–793. doi:10.14778/3636218.3636232
- [9] Jian Zhou, Luna Wang, Shuaihua Zhao, Chen Zhong, and Song Jiang. 2026. LiBox: A Learned Index as an Array to Minimize Last-Mile Search. *Proc. VLDB Endow.* 19, 5 (2026), 836–848. <https://www.vldb.org/pvldb/vol19/p836-jiang.pdf>